

OCA I2C Controller — Test Plan

SMC I2C Controller — Detailed Test Plan

Contents

1. Objectives
2. Scope (in / out)
3. Verification strategy
4. Environment
5. Test bench architecture
 - 5.1 `i2c_controller_tb.cpp` (primary)
 - 5.2 `i2c_controller_neg_tb.cpp` (negative / edge)
6. Stimulus and checking primitives
7. Feature to test traceability matrix
8. Detailed test cases
 - 8.1 Primary bench (`i2c_controller_tb`)
 - 8.2 Negative bench (`i2c_controller_neg_tb`)
9. Coverage plan
10. Negative-test catalogue
11. Regression workflow
12. Pass / fail criteria and exit conditions
13. Future tests and open work

SMC I2C Controller — Detailed Test Plan

Document: 03_I2C_CONTROLLER_Test_Plan.md **Module under test (MUT):** smc::i2c_controller (peripherals/i2c_controller/) **Reference test benches:** test/i2c_controller_tb.cpp, test/i2c_controller_neg_tb.cpp **RTL reference:** hw/comp/i2c/ and hw/comp/i2c/data/registers/rd1/i2c.rdl **Status:** Implemented — both benches pass; line coverage of src/i2c_controller.cpp > 95 % **Companion docs:** - 01_I2C_CONTROLLER_Specification.md — externally-observable behaviour - 02_I2C_CONTROLLER_LowLevel_Design.md — implementation contract

Contents

1. Objectives
 2. Scope (in / out)
 3. Verification strategy
 4. Environment
 5. Test bench architecture
 6. Stimulus and checking primitives
 7. Feature to test traceability matrix
 8. Detailed test cases
 9. Coverage plan
 10. Negative-test catalogue
 11. Regression workflow
 12. Pass / fail criteria and exit conditions
 13. Future tests and open work
-

1. Objectives

The I2C test plan demonstrates that the SystemC functional model:

1. **Conforms** to the programming model in 01_I2C_CONTROLLER_Specification.md and the ground-truth i2c.rdl register map (offsets, field masks, access types).
 2. **Implements correctly** every flow described in the specification: Controller-Mode write/read transactions, repeated START, NACK/halt/resume, NAKOK, Target-Mode ACQ/TX FIFOs and address matching, FIFO thresholds and resets, the 20-source interrupt block, and INTR_TEST force paths.
 3. **Honours the TLM-2.0 contract:** response codes, transport_dbg side-effect-free reads, no DMI, LT delay annotation driven by a quantum keeper.
 4. **Is robust** against malformed accesses and boundary conditions (bad width/alignment/ command, decode misses, FIFO overflow, immutable-param writes).
-

2. Scope (in / out)

In scope

- All decoded registers (`0x00..0x80`), their access types and field masks.
- Controller-Mode datapath through a test-bench bus model.
- Target-Mode datapath through the `target_write` / `target_read` back doors.
- Interrupt aggregation (level, latched/W1C, forced) and `irq_o` behaviour.
- FIFO occupancy, thresholds, resets, and overflow error interrupts.
- TLM error responses, `transport_dbg`, reset.
- CCI parameter presets, mutability, and introspection.

Out of scope (see `01_* §13` / `02_* §15`)

- Bit-level SCL/SDA waveform, clock stretching, arbitration, glitch filtering.
- Analog timing (`TIMINGx`) and timeout firing.
- SMBus alert/suspend signalling and multi-controller monitor behaviour.

3. Verification strategy

- **Self-checking C++ benches** built on SystemC + CCI; each prints `ALL TESTS PASSED` and returns 0 only if every `EXPECT_*` passed. `sc_report` errors are also failure signals.
- **Directed tests** cover each spec flow with deterministic stimulus and golden checks.
- **Bus-model injection** emulates remote target(s) for Controller-Mode segments, letting a single bench script ACK/NACK and supply read data arbitrarily.
- **Back-door target driving** exercises Target Mode without a second controller model.
- **A dedicated negative bench** runs entirely during elaboration (function calls through the bound socket, no `sc_start`) so constructor-fatal probes and error-response checks are isolated from a running kernel.
- **Coverage-guided**: the two benches together drive `src/i2c_controller.cpp` above 95 % line coverage; gaps are closed with targeted cases (repeated START, resume-with-pending, overflow, dbg peeks).

4. Environment

Item	Value
Language standard	C++20 (per repo <code>cpp20-build</code> rule)
SystemC	Accellera 3.0.x (compatible with $\geq 2.3.4$)
CCI	Accellera CCI 1.0
Build	CMake; <code>run_tests.sh</code> wraps <code>configure/build/run</code>
Sanitizers	<code>--asan</code> (ASan/LSan)
Coverage	<code>--coverage</code> (llvm-cov or gcovr/lcov)
CTest	<code>--ctest</code> ; <code>PASS_REGULAR_EXPRESSION = "ALL TESTS PASSED"</code>

5. Test bench architecture

5.1 i2c_controller_tb.cpp (primary)

- `sc_main` registers a global CCI broker and injects presets before construction: `tb.i2c.rx_fifo_depth = 8`, `access_delay_ns = 5`, `xfer_delay_ns = 20` (short latency).
- A driver `sc_module` owns a `tlm_utils::tlm_quantumkeeper`; `read32/write32` seed the payload delay from the QK, call `b_transport`, then `set_and_sync`. `dbg_read/dbg_write` exercise `transport_dbg`.
- A `tb` root binds the driver socket to `dut.reg_socket`, wires `rst_n/irq`, and installs a bus model that records the last segment and returns programmable ACK/read-data.
- An `SC_THREAD` (`run`) drives the directed sequence, using `step()` to advance real time so the `xfer_event_` fires, and `pulse_reset()` between independent cases.

5.2 i2c_controller_neg_tb.cpp (negative / edge)

- Sets `SC_FATAL` → `SC_THROW` so constructor guard rails can be probed with `expect_fatal`.
- Constructs a DUT with tiny FIFOs (depth 2) to reach overflow paths quickly.
- Drives raw payloads through a `probe` initiator; never calls `sc_start` (elaboration-only), so unbound `rst_n_i/irq_o` are harmless.

6. Stimulus and checking primitives

Primitive	Purpose
<code>EXPECT_EQ(exp, act) / EXPECT_TRUE(cond)</code>	Record a failure (and message) without aborting.
<code>driver::read32/write32</code>	Timed register access through the quantum keeper.
<code>driver::dbg_read/dbg_write</code>	<code>transport_dbg</code> access (no delay, no side effects on read).
bus model lambda	Emulate the remote target: capture the segment, set <code>ack</code> , supply <code>read_data</code> .
<code>dut.target_write/target_read</code>	Drive Target Mode as an external controller.
<code>dut.dbg_*_count()</code> / <code>dbg_reg()</code> / <code>dump_state()</code>	Observe internal state for checks.
<code>step()</code> / <code>settle()</code> / <code>pulse_reset()</code>	Advance time for timed xfers; flush deltas; reset.

7. Feature to test traceability matrix

Spec feature	Test case(s)
Reset values (regs, STATUS, irq)	TB-1
Register R/W + field masking	TB-2
Controller write transaction	TB-3
Controller read transaction (RX FIFO)	TB-4
RX FIFO overflow	TB-4b
Trailing open segment	TB-4c
CMD_COMPLETE interrupt + W1C	TB-5
NACK halt + resume	TB-6, TB-10d
NAKOK suppresses halt	TB-7
FMT threshold interrupt	TB-8
RX threshold interrupt + RXRST	TB-9
FMT reset (FIFO_CTRL)	TB-10
Enable-host-after-queue kick	TB-10b
Repeated START → two segments	TB-10c
Target write / ACQ FIFO + START/STOP detect	TB-11
Target read / TX FIFO	TB-12
Target NACK count (saturate + read-clear)	TB-13
ACQ / TX threshold interrupts	TB-14
INTR_TEST force (latched + level)	TB-15
TARGET_ACK_CTRL / ACQ_FIFO_NEXT_DATA	TB-16
transport_dbg + dbg_reg	TB-17
CCI introspection / preset / mutable	TB-18
Constructor guard rails (zero FIFO depth)	NEG-1
TLM error responses	NEG-2
RO write-ignore / WO read-as-zero	NEG-3
FIFO overflow errors (FMT/TX/ACQ)	NEG-4..6
Disabled-target back door NACK	NEG-7
Direct TARGET_NACK_COUNT + TARGET_EVENTS W1C	NEG-8
transport_dbg malformed + dbg peeks	NEG-9
CCI immutability	NEG-10

8. Detailed test cases

8.1 Primary bench (i2c_controller_tb)

- **TB-1 Reset values** — after `pulse_reset`, `INTR_STATE`/`INTR_ENABLE`/`CTRL` read 0, `irq` low, `STATUS` = `FMTEEMPTY`|`HOSTIDLE`|`TARGETIDLE`|`RXEMPTY`|`TXEMPTY`|`ACQEMPTY`.
- **TB-2 Register R/W + masking** — `CTRL`, `OVRD`, `TIMING0`, `TIMEOUT_CTRL`, `HOST_NACK_HANDLER_TIMEOUT`, `SMBUS_CTRL` mask to their field widths; `VAL`/`SMBUS_STATUS` read 0 and ignore writes.
- **TB-3 Controller write** — enable host, push `{START, addr}`, `data`, `{data, STOP}`; the bus model sees one write segment with the two data bytes; FMT drains to empty.
- **TB-4 Controller read** — request 3 bytes; bus model returns them; they land in the RX FIFO and read back in order through `RDATA`; an extra read returns 0.
- **TB-4b RX overflow** — request 10 bytes into an 8-deep RX FIFO; occupancy clamps to 8 and `RX_OVERFLOW` latches.
- **TB-4c Trailing open segment** — a write segment with no STOP still executes.
- **TB-5 CMD_COMPLETE** — enabled, fires on STOP, raises `irq`; W1C clears it and drops `irq`.
- **TB-6 NACK halt + resume** — bus NACKs; `CONTROLLER_EVENTS.NACK` set, `CONTROLLER_HALT` raised; W1C the events → resumes, halt clears, `irq` drops.
- **TB-7 NAKOK** — a NACK with NAKOK does not halt; STOP still completes.
- **TB-8 FMT threshold** — `FMT_THRESH=4`, FMT empty ($0 < 4$) → interrupt + `irq`.
- **TB-9 RX threshold + RXRST** — `RX_THRESH=1`, read 4 bytes ($4 > 1$) → interrupt; `HOST_FIFO_STATUS` shows `FMTLVL=0/RXLVL=4`; `RXRST` empties RX.
- **TB-10 FMT reset** — host disabled, push 2 entries, `FMTRST` empties FMT.
- **TB-10b Enable-after-queue** — queue FMT with host off, then set `ENABLEHOST` → engine kicks and drains.
- **TB-10c Repeated START** — one FMT stream with two STARTs (no intervening STOP) → two bus segments; last address observed.
- **TB-10d Resume with pending FMT** — first segment NACK/halts with a second segment still queued; clearing events drains the second.
- **TB-11 Target write** — matched `target_write` fills ACQ with Start + data + Stop; entries read via `ACQDATA` with correct SIGNAL; START/STOP detect latch in `TARGET_EVENTS`.
- **TB-12 Target read** — fill TX via `TXDATA`; `target_read` drains it; `TARGET_FIFO_STATUS` reflects TX level.
- **TB-13 Target NACK count** — unmatched write+read bump the saturating counter to 2; a read clears it (read-clear).
- **TB-14 ACQ/TX thresholds** — thresholds cross → ACQ/TX threshold interrupts + `irq`.
- **TB-15 INTR_TEST force** — pulse-force a latched source (`SMBALERT`), W1C clears; hold a level source (`FMT_THRESHOLD`) while the test bit is set.
- **TB-16 TARGET_ACK_CTRL** — NBYTES read-back; the NACK pulse bit bumps the NACK count; `ACQ_FIFO_NEXT_DATA` reads 0 when ACQ empty.
- **TB-17 transport_dbg** — a debug read of `ACQDATA` does not pop the FIFO; a debug write of `TARGET_ID` takes effect; `dbg_reg` matches.
- **TB-18 CCI introspection** — preset `rx_fifo_depth` reads 8 [preset]; default `fmt_fifo_depth` reads 64; `access_delay_ns` is mutable; all handles enumerated.

8.2 Negative bench (i2c_controller_neg_tb)

See §10.

9. Coverage plan

- **Target:** > 95 % line coverage of `src/i2c_controller.cpp` (achieved: ~99 %).
- **Method:** `./run_tests.sh --coverage` merges both benches' profiles and prints a per-file report plus the uncovered-line list for the source.
- **Closed gaps:** repeated-START execute (TB-10c), enable-after-queue schedule (TB-10b), resume-with-pending schedule (TB-10d), RX overflow (TB-4b), FMT/TX/ACQ overflow (NEG-4..6), every `dbg_reg` branch and the regmap-miss fallback (NEG-9), `TARGET_EVENTS` W1C (NEG-8).
- **Residual:** bit-level/timeout/SMBus paths are intentionally out of scope and are exercised (where reachable) only through `INTR_TEST`.

10. Negative-test catalogue

ID	Case	Expected
NEG-1	Construct with a zero FIFO depth (fmt / acq)	<code>SC_REPORT_FATAL</code> (caught)
NEG-2a	Read width 2 or 8	<code>TLM_BURST_ERROR_RESPONSE</code>
NEG-2b	Misaligned address (<code>CTRL+2</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
NEG-2c	Out-of-window (<code>WINDOW_SIZE</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
NEG-2d	In-window decode miss (<code>0x84</code>) read/write	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
NEG-2e	<code>TLM_IGNORE_COMMAND</code>	<code>TLM_COMMAND_ERROR_RESPONSE</code>
NEG-3	Write RO regs (<code>STATUS/HOST_FIFO_STATUS/ACQ_FIFO_NEXT_DATA</code>)	ignored; read WO regs → 0
NEG-4	FMT overflow (3rd into depth-2)	<code>CONTROLLER_TX_FIFO_ERROR</code>
NEG-5	TX overflow	<code>TARGET_TX_FIFO_ERROR</code>
NEG-6	ACQ overflow (Start+data+Stop into depth-2)	<code>TARGET_RX_FIFO_ERROR</code>
NEG-7	Back door with <code>ENABLETARGET=0</code>	write/read return false (NACK)
NEG-8	Direct <code>TARGET_NACK_COUNT</code> write; <code>TARGET_EVENTS</code> W1C	round-trip; cleared
NEG-9	<code>transport_dbg</code> malformed (align/width/OOB/bad cmd/write-miss); <code>dbg</code> peeks	0 for malformed; 4 + no-clear for peeks
NEG-10	Set an immutable param after construction	rejected (throw or no-op)

11. Regression workflow

```
# From peripherals/i2c_controller/
./run_tests.sh                # Release build + run both benches
./run_tests.sh --ctest        # via CTest
./run_tests.sh --asan         # AddressSanitizer/LeakSanitizer
./run_tests.sh --coverage     # line-coverage report

# From smc/ (whole-IP pipeline: Release / ASAN / Coverage / CTest)
./run_all_smc_tests.sh i2c_controller
```

`i2c_controller` is registered in `smc/run_all_smc_tests.sh`, so the nightly/CI `run_all` sweep builds and runs it alongside the other IPs.

12. Pass / fail criteria and exit conditions

- **Pass:** both benches print `ALL TESTS PASSED` and exit 0; CTest matches the pass regex; ASan reports no errors; coverage of the source file is > 95 %.
- **Fail:** any `EXPECT_*` failure (prints `FAIL <file>:<line> ...`), a non-OK response on a legal access, an unexpected `sc_report` error, a sanitizer finding, or a coverage regression below the threshold.

Note: the ASan link step requires the 64-bit `libasan` runtime; on a host lacking it the ASan stage fails to link for every SMC IP (environmental, not a model defect). CI runners provide the runtime.

13. Future tests and open work

- Arm `TIMEOUT_CTRL` / host/target/NACK-handler timeouts and test the corresponding interrupts once timeout firing is modelled.
- Add SMBus alert/suspend functional tests if `SMBUS_CTRL` / `SMBUS_STATUS` gain behaviour.
- Multi-controller / monitor-mode observation tests when that path is modelled.
- Randomised/constrained transaction sequences (mixed read/write, repeated START chains) layered on the directed suite for additional confidence.